GATEFLOW AI

School-Gate Traffic Digital Twin
and Decision-Support System

Complete solution • Technical workflow • Mathematical formulation • 24-hour MVP plan

A school-focused planning platform that evaluates staggered class timings and pickup-zone assignments, then tests candidate plans in a microscopic traffic simulation before implementation.

DESIGNED FOR A 24-HOUR HACKATHON MVP

Core principle

GateFlow AI is a decision-support tool, not an automatic traffic controller. It compares feasible options under explicit assumptions, exposes trade-offs and uncertainty, and leaves final approval to school administrators.

Scope of this document

This document defines the recommended merged solution: a complete product workflow, uncertainty-aware demand estimation, constrained multi-objective schedule planning, pickup-zone capacity checks, SUMO-based vehicle simulation, stress testing and a reproducible demonstration strategy. Advanced features are clearly separated from the 24-hour MVP.

Contents

1. Executive summary and problem definition
2. Product boundaries and operating assumptions
3. End-to-end operational workflow
4. Mathematical formulation and formulas
5. Technical architecture and recommended tools
6. Data requirements and uncertainty handling
7. 24-hour implementation plan
8. Validation, metrics and demonstration strategy
9. Scope control: what to build and defer
10. Final assessment: uniqueness, judging potential and feasibility
11. Final pitch to evaluators

Reading note: Formulas are prototype-level definitions intended to make the planning logic explicit. Their exact implementation and interpretation must be tested against the selected network, available observations and simulation outputs.

1. Executive summary and problem definition

GateFlow AI is an uncertainty-aware school-gate traffic planning platform. It searches for better class arrival/departure schedules, assigns suitable pickup zones, and tests candidate plans in a microscopic traffic simulation before a school implements them.

The immediate operational problem is that a school must coordinate two limited resources: **time** and **kerbside stopping space**. When too many vehicles arrive together, queues can exceed the available stopping area. However, simply moving every class to a different time may be unacceptable: siblings may attend different classes, some grades may have fixed timings, families have work commitments, and buses need safe access.

GateFlow frames this as a constrained planning problem:

> **Find a feasible combination of class timings and pickup-zone assignments that reduces simulated queueing and spillback, while limiting family disruption and remaining robust to uncertain traffic demand.**

The defining combination

- School-specific hard constraints and operational preferences.
- Data-driven or explicitly assumption-based estimates of vehicle demand.
- Multi-objective schedule optimisation that makes trade-offs visible.
- Pickup-zone capacity checks and assignment rules.
- Microscopic vehicle-level evaluation using SUMO.
- Stress testing under demand changes, longer stopping times and imperfect compliance.
- An explainable plan and parent notice for principal review.

What GateFlow does not claim

- It cannot create additional road space or guarantee that families will follow the schedule.
- It does not establish that a road segment is legally or safely usable for pickup just because it appears on a map.
- A simulation is not proof of real-world traffic reduction. Real-world benefit must be measured through a school pilot.
- Synthetic data can demonstrate algorithm behaviour under stated assumptions, but cannot establish real-world effectiveness.

2. Product boundaries and operating assumptions

The recommended hackathon scope is deliberately narrow: one school, one gate, one morning or afternoon peak, and a small road network. This is enough to demonstrate the full decision loop without making integration risk unmanageable.

Item	MVP definition
Primary user	School principal or transport coordinator
School groups	Six fictional student groups for the reproducible demo
Network	One gate, one road approach, an adjacent junction and a few pickup bays
Traffic data	Manual observations if available; otherwise clearly labelled synthetic data or configurable assumptions
Optimisation	Discrete schedule offsets; hard constraints enforced; congestion/disruption trade-offs shown
Pickup zones	Three or four candidate zones, with capacities and safety/legal status verified by the responsible school/authority
Simulation	SUMO vehicle-level runs for baseline and shortlisted candidate plans
Outputs	Candidate schedules, zone assignments, queue/delay metrics, stress-test results and an approved-plan notice
Deployment	Local laptop/localhost; no paid API is required for the core workflow

Operating assumptions to show in the interface

- Every demand input is labelled as observed, synthetic or assumed.
- The model uses a defined simulation period and time-step.
- Vehicle types, vehicle lengths, stopping durations and route choices are scenario inputs—not universal facts.
- Candidate pickup zones require human verification for legal use, safe pedestrian access, emergency access and actual usable kerb length.
- The fast planning model is used to shortlist schedules; SUMO results are used to evaluate those shortlisted plans in more detail.
- If the fast model and SUMO disagree, the dashboard must show the discrepancy and use the simulation results for final candidate comparison.

Two-level evaluation strategy

Level	Purpose	Typical output
Level 1 — Fast queue model	Evaluate many schedule combinations quickly, estimate queueing and disruption, reject infeasible plans.	Feasible shortlist and trade-off ranking
Level 2 — Microscopic simulation	Model individual vehicles and evaluate shortlisted plans on a small road network.	SUMO queue, delay, waiting and spillback metrics

3. End-to-end operational workflow

The workflow below follows the intended product journey from a reproducible demo setup through planning, simulation, approval and optional follow-up measurement.

Phase 0 — Prepare the demonstration scenario

Prepare one reproducible school scenario before the principal enters information. Use a small road network near a selected school if suitable OpenStreetMap data is available; otherwise create a generated SUMO network with a school gate, road approach, adjacent junction and pickup bays. Keep a saved network snapshot so the demo does not depend on a last-minute map download. Include six fictional student groups, an existing bell schedule, vehicle-type proportions, gate capacity and three or four pickup-zone candidates.

Output: A baseline scenario that can be run repeatedly.

Phase 1 — Principal configures the school

The dashboard collects student counts by group, current arrival/departure times, permitted changes, vehicle proportions and gate-side capacity. A map shows the school and nearby roads. The user can mark candidate stopping areas or review prepared candidates. A capacity checker estimates whether the stopping space may be sufficient.

Output: A validated school-configuration record, stored as JSON for the MVP.

Phase 2 — Build a traffic-demand profile

The user selects manual gate observations, a prepared synthetic dataset or configurable assumptions. For a manual 20-minute observation, record four five-minute bins for each vehicle type and sample stopping durations. The calibration engine estimates arrival rates and vehicle shares while retaining uncertainty. Without observations, configurable priors are used and the output is labelled assumption-based. The system generates plausible vehicle arrival times around each group's schedule; vehicle type, occupancy, length and stopping duration affect demand.

Output: A time-dependent demand profile with source labels, assumptions and uncertainty.

Phase 3 — Diagnose the existing arrangement

Simulate the current schedule before proposing changes. Record peak queue length, waiting-time distribution, total vehicle delay, spillback duration, idling vehicle-minutes and pickup-zone capacity violations. Save the demand assumptions and random seed so baseline and proposed plans can be compared fairly. Display the current schedule, queue-over-time chart and simulation replay when available.

Output: A baseline against which every proposed plan is evaluated.

Phase 4 — Define constraints

The principal enters restrictions such as a group that cannot start before a given time, a group whose timing must remain fixed, sibling groups that need closely aligned schedules, bus access to a designated bay, and a maximum acceptable timing change. Use a structured form for the MVP. An optional LLM may convert free text into structured rules, but the validated backend remains authoritative. If rules conflict, report the conflict before optimisation.

Output: Validated hard constraints and soft preferences. Hard constraints must not be traded away for a lower queue score.

Phase 5 — Search for alternative schedules

Test discrete shifts for six groups. Example offsets are –15, –10, –5, 0, +5, +10 and +15 minutes. Six groups with seven choices each yield 7^6 = 117,649 combinations before constraints. Use a fast queue model to evaluate combinations cheaply, discard infeasible schedules and present candidates across congestion/disruption trade-offs rather than one unexplained answer.

Output: A set of feasible candidate schedules.

Phase 6 — Assign vehicle types to pickup zones

Consider vehicle types and feasible stopping areas for each candidate. Nearby road data may suggest candidates, but the school or responsible authority must verify legality, safety, pedestrian access, emergency access and usable kerb length. Estimate capacity using usable length and vehicle-spacing assumptions. Reject assignments that exceed capacity or violate rules.

Output: Timings, vehicle-type assignments, capacity warnings and the zones represented in the simulation.

Phase 7 — Evaluate shortlisted plans in SUMO

Pass a shortlist of feasible plans into SUMO. Model individual vehicles travelling through the school-zone network, with demand reflecting timings, vehicle types, routes and stopping behaviour. Record simulated queues and delays. Use consistent scenario seeds where practical to avoid unfair comparisons from different random arrival patterns. Re-rank candidates using simulation performance and distinguish fast-model estimates from SUMO results.

Output: A model-evaluated shortlist with comparable simulation metrics.

Phase 8 — Stress-test the plans

Re-run shortlisted plans under normal demand, a demand surge, configurable rainy-day assumptions, 60% compliance with new timing windows, and longer loading/unloading duration. Unless based on observations, these are synthetic scenarios. Compare averages, high-percentile waiting times and worst outcomes observed in the simulation sample.

Output: A robustness comparison and explicit description of conditions where a plan may fail.

Phase 9 — Principal compares and chooses

Show three or four options: Minimal change (prioritises family routine), Balanced (trade-off between congestion and disruption), Congestion-focused (lower simulated queues within constraints), and Keep as is (baseline). These labels describe selection criteria; they do not guarantee real-world outcomes. The principal reviews schedules, zone map, stress tests and uncertainty before selecting a candidate.

Output: A human-reviewed candidate plan, with uncertainty and limitations visible.

Phase 10 — Generate a parent notice

After selection, an AI co-pilot may draft an English or Tamil notice using the approved schedule and zone assignment. The numerical values must come from the selected plan's stored data. A deterministic verifier checks that times and locations match the approved plan, and the principal reviews the notice before release.

Output: An actionable plan and verified parent notice.

Phase 11 — Optional follow-up measurement

After trialling a plan, staff collect another gate count and compare observations against the baseline and predictions. If observed congestion falls outside the expected range, flag the mismatch and update estimates. For the hackathon, use a second synthetic dataset; in real deployment, actual counts are required to assess real-world impact.

Output: A feedback loop that can learn from observed deviations instead of permanently trusting an unvalidated model.

4. Mathematical formulation and formulas

The formulas below define the main quantities used by the prototype. They are deliberately transparent and can be implemented incrementally. The exact queue model, parameter estimates and simulator outputs should be validated against test scenarios.

4.1 Schedule search space

$$\text{Number of schedules before constraints} = K^G$$

K = number of permitted timing offsets per group; G = number of student groups.

For six groups and seven offsets (–15, –10, –5, 0, +5, +10, +15 minutes):

$$7^6 = 117{,}649 \text{ candidate schedules}$$

The constraint validator reduces the feasible search space before more expensive simulation.

4.2 Approximate total vehicle waiting time

$$J_{delay} = \sum_t Q_t \times \Delta t$$

Q_t = estimated number of waiting vehicles during time interval t; Δt = interval duration in minutes. This approximates total vehicle waiting time in vehicle-minutes.

4.3 Schedule-disruption proxy

$$J_{disruption} = \sum_g N_g \times |\delta_g|$$

N_g = group-size or family-count proxy for group g; δ_g = timing shift in minutes. If N_g is student count, this is a weighted student-minute proxy—not a literal count of affected families.

4.4 Combined score versus Pareto trade-off

The preferred approach is to show the Pareto trade-off between congestion and disruption. If a single ranking score is needed for a simple MVP, normalise both objectives first:

$$J_{combined} = \alpha \times J_{delay_norm} + \beta \times J_{disruption_norm}$$

α and β are administrator-selected weights; $\alpha + \beta = 1$ for a simple weighted score. Normalisation is important because the two objectives have different units. A weighted score is a convenience, not a substitute for showing the trade-off.

A plan is Pareto-dominated if another feasible plan is no worse on both objectives and strictly better on at least one. Keep non-dominated candidates so the principal can choose based on operational priorities.

4.5 Arrival-rate estimate from manual counts

$$\lambda\blacksquare = C / T$$

C = number of observed vehicle arrivals; T = observation duration in minutes; $\lambda\blacksquare$ = estimated arrivals per minute. For a 20-minute observation, use four five-minute bins and calculate the rate per bin or over the full period.

A basic Bayesian count model can use a Gamma prior for an unknown Poisson arrival rate:

$$\lambda \sim \text{Gamma}(a, b);\ C \mid \lambda \sim \text{Poisson}(\lambda T)$$

Here b is the rate parameter. After observing C arrivals over duration T, the posterior is λ | C ~ Gamma(a + C, b + T), using the rate-parameter convention.

This gives a distribution over arrival rates rather than a single fixed number. If no observed counts exist, prior-based results must be labelled assumption-based.

4.6 Vehicle-type mix

$$p = (p_1, p_2, \ldots, p_m), \Sigma_i p_i = 1$$

p_i is the estimated proportion for vehicle type i. A Dirichlet model can represent uncertainty in the vehicle mix; update it with observed counts by adding each type's count to its prior concentration parameter.

4.7 Pickup-zone capacity estimate

$$C_{zone} = \text{floor}(L_{usable} / (L_{vehicle} + S_{gap}))$$

L_usable = verified usable kerb length; L_vehicle = assumed average stopping length for the relevant vehicle type; S_gap = spacing allowance. This is a simplified estimate and must not replace safety, legal or engineering checks.

4.8 Queue evolution in the fast model

$$Q_{t+1} = \max(0, Q_t + A_t - \mu_t)$$

Q_t = queue at time t; A_t = arrivals during the interval; μ_t = vehicles that can complete the stopping/service process in that interval. The model needs consistent units and a defined service-capacity assumption.

4.9 Waiting-time distribution and robustness

$$W_{95} = 95\text{th percentile of simulated vehicle waiting times}$$

Use the empirical 95th percentile across simulated vehicles/runs to represent a high-wait outcome. Report the sample size and scenario assumptions.

$$J_{robust} = \text{mean}(J \text{ across scenarios}) \text{ and } J_{worst} = \max(J \text{ across tested scenarios})$$

These are simple robustness summaries. More advanced tail-risk measures such as CVaR are optional and should only be added if implemented correctly.

4.10 Idling and delay metrics

$$\text{Vehicle delay} = \Sigma_i (T_{actual,i} - T_{reference,i})$$

Define the reference travel time consistently. Idling vehicle-minutes can be estimated by summing vehicle idle durations, if the simulator output supports the required distinction.

4.11 Scenario comparison

$$\Delta\% = 100 \times (\text{Baseline} - \text{Candidate}) / \text{Baseline}$$

Use only when the baseline metric is greater than zero and both runs use comparable assumptions. Label this as a simulated percentage change, never as a guaranteed real-world reduction.

Implementation rule: Save the baseline configuration, scenario settings, random seed, model version and metric definitions with every run. If these differ between candidates, the comparison may not be fair.

5. Technical architecture and recommended tools

The core planning engine should run locally. The LLM is optional and should not be responsible for the mathematical optimisation or for deciding whether a zone is safe.

Layer	Recommended tool	Role in GateFlow
Web interface	React + Vite + Tailwind CSS	School setup, constraint form, dashboard and plan selection
Backend	Python + FastAPI + Pydantic	APIs, schema validation, orchestration and result storage
Geospatial data	OpenStreetMap + OSMnx	Road-network extraction and analysis where data is available
Map display	Leaflet	School, road segments and candidate pickup zones
Fast planning model	NumPy + SciPy	Queue approximation, candidate evaluation and optimisation calculations
Bayesian calibration	NumPy/SciPy distributions	Update arrival-rate and vehicle-mix estimates from observations
Microscopic simulation	SUMO	Vehicle movement, queues, stopping behaviour and delays
Optional optimiser	pymoo / NSGA-II	Evolutionary multi-objective search if the basic search needs it
Charts	Plotly	Queue-over-time charts, plan comparison and uncertainty displays
Storage	JSON and CSV	School configuration, counts, scenarios, plans and results
Optional local LLM	Ollama + suitable local model	Translate free-text constraints or draft explanations/notices

Suggested logical architecture

Component	Receives	Produces
React dashboard	User inputs and plan choice	Validated requests and visual comparisons
FastAPI validation layer	School profile, constraints, counts	Validated configuration; conflict/error messages
Demand estimator	Observed/synthetic counts and priors	Time-dependent demand profile and uncertainty
Fast planner	Demand, constraints, zone capacities	Feasible schedules and trade-off shortlist
SUMO adapter	Shortlisted schedules, routes and scenarios	Vehicle-level simulation metrics and replay data
Robustness evaluator	Metrics from multiple scenarios	Average, high-percentile and worst-sampled outcomes
Notice generator + verifier	Approved plan JSON	Draft notice and exact-value validation result

Free-tool and reliability guidance

- The MVP can run locally without paid software, paid LLM APIs, sensors or paid traffic-data APIs.
- OpenStreetMap is open data, but standard tile servers have usage rules; provide attribution, respect caching policies and avoid bulk tile downloads.
- Keep a saved SUMO network and sample demand file so the demo can run without a live download.
- A structured constraint form and template-based notice are acceptable fallbacks if LLM integration is unreliable.
- Verify the selected local model's licence and whether the laptop can run it. Free software does not mean unlimited compute or unlimited external API access.

6. Data requirements and uncertainty handling

GateFlow uses three categories of data. Keeping these categories visible is essential to technical credibility.

Category	Examples	Limitations
Open geographic data	Road geometry and available road attributes	Does not guarantee live traffic, legal pickup permissions or verified kerb availability
Entered/observed data	Student counts, bell times, vehicle mix, manual arrivals, stopping durations, usable stopping space	May be incomplete or noisy; observations should be time-stamped and scoped to a specific period
Synthetic data	Vehicle arrival sequences, dwell-time variation, compliance, demand surge and stress cases	Useful for reproducible demos but cannot prove real-world benefit

Manual observation protocol

- Choose one defined arrival or departure period.
- For a 20-minute observation, record four consecutive five-minute bins.
- Count arrivals separately by vehicle type where feasible.
- Sample stopping/loading durations rather than assuming all vehicles stop for the same time.
- Record date, weather/context, observation period and known disruptions.
- If observations are too limited to estimate uncertainty reliably, state this clearly and use broader scenario ranges.

Uncertainty-aware demand estimation

Arrival rates can be represented by a Gamma distribution, vehicle-type proportions by a Dirichlet distribution, and stopping durations by an appropriate positive-valued model. A more advanced implementation may use a Normal-Inverse-Gamma model for log stopping duration. These are modelling options, not mandatory components for the first demo.

Use uncertainty to generate multiple plausible demand scenarios, rather than displaying one apparently precise forecast. A useful dashboard should show whether a plan remains acceptable across the tested range and should identify where the result becomes fragile.

Data integrity rules

- Mark each result as observed-data-based, synthetic or assumption-based.
- Store the exact inputs and random seed used for every run.
- Do not infer legal/safe pickup status from OpenStreetMap alone.
- Do not report simulated results as measured real-world improvements.
- Do not describe student counts as family counts unless household-level data actually supports that interpretation.

7. Realistic 24-hour implementation plan

The first milestone is not a polished dashboard. It is a complete, repeatable run from school inputs to a computed result. Test SUMO early because network setup and output extraction are major integration risks.

Time	Priority	Deliverable / acceptance check
Hours 0–2	Lock scope and test SUMO	Install dependencies, run one tiny simulation, read vehicle outputs and prepare a fallback network. Freeze the six-group data schema.
Hours 2–5	School setup and inputs	Configuration, constraints, manual counts, sample-data mode and synthetic demand generation.
Hours 5–9	Baseline simulation pipeline	Generate baseline SUMO scenario, routes, queue/delay metrics and reproducible saved runs.
Hours 9–13	Schedule planner	Fluid queue model, allowed offsets, hard-constraint validation and congestion/disruption ranking.
Hours 13–16	Zones and stress tests	Zone capacity checks, prohibited assignment rejection, increased demand and reduced-compliance scenarios.
Hours 16–20	Dashboard integration	Baseline-versus-candidate comparison, queue chart, candidate-zone map, uncertainty and plan selection.
Hours 20–22	Notice and quality checks	Generate parent notice, verify numerical values, test edge cases and run the end-to-end workflow.
Hours 22–24	Freeze and rehearse	Repeat demo, check dependencies, record backup video and prepare concise technical explanations.

Recommended team parallelism

- Simulation/data owner: SUMO network, vehicle routes, scenario inputs and metrics.
- Optimisation owner: queue model, constraints, candidate schedules and objective values.
- Frontend/integration owner: React dashboard, API wiring, charts and plan selection.
- One shared JSON schema and metric definitions must be agreed early so the parts integrate.

Cut line if the team falls behind

Keep: school form, constraint validation, baseline simulator, schedule recommendations, before/after comparison and at least one stress test.
Defer: advanced Bayesian calibration, local LLM, Gaussian-process surrogate, NSGA-II, CVaR and multi-school features.

A fully functioning system with a transparent algorithm is better than a visually impressive prototype whose recommendations never complete the workflow.

8. Validation, metrics and demonstration strategy

The most convincing demo shows a decision being made, not a list of AI technologies. Every result should be reproducible and tied to the assumptions used.

Recommended baseline comparisons

Plan	Purpose
Existing schedule	Establish the current simulated baseline
Uniform stagger	Provide a simple, understandable benchmark
GateFlow candidate	Show the effect of constrained, multi-objective planning

Metrics to display

- Peak queue length (vehicles).
- Mean and 95th-percentile waiting time (minutes).
- Total vehicle delay (vehicle-minutes).
- Spillback duration (minutes), if represented and measured in the network.
- Idling vehicle-minutes, if supported by the output data.
- Pickup-zone capacity violations.
- Schedule-disruption proxy and exact objective values.
- Performance across normal and stress scenarios.

Six-step judge demonstration

Step	What to show	What it proves
1. Baseline	Run the existing timetable; show queue growth and capacity pressure.	The problem is represented in a working model.
2. Constraints	Freeze one group's timing and limit another group's shift.	Hard rules are enforced, not ignored.
3. Search	Generate candidates and plot congestion versus disruption.	The planner explores real trade-offs.
4. SUMO replay	Run shortlisted options and show actual simulated metrics and replay.	The traffic output is produced by a simulator, not a fabricated graphic.
5. Stress test	Increase demand or reduce compliance to 60%.	The team evaluates robustness and exposes failure conditions.
6. Action	Select a plan and generate a verified parent notice.	The workflow ends in a usable decision artifact.

Five things judges should be able to inspect

- **Reproducibility:** saved configuration, fixed seeds and one-click demo.
- **Honest baseline:** existing schedule versus uniform stagger versus GateFlow candidate.
- **Real trade-off:** why a minimal-change plan differs from a congestion-focused plan.
- **Credible simulation:** explain that the network is simplified and demand is synthetic unless observations were collected.
- **Failure handling:** an infeasible constraint or over-capacity zone is rejected or clearly flagged.

Technical claims to make only if implemented

- Bayesian calibration updates uncertain arrival-rate and vehicle-mix estimates from observations.
- Multi-objective search finds schedules balancing delay and family disruption.
- SUMO evaluates shortlisted plans at vehicle level.
- Scenario testing examines robustness to demand and compliance changes.
- A deterministic verifier checks final notice values against the approved plan.

Strongest technical claim: GateFlow evaluates recommendations against explicit baselines, under stated constraints and repeatable traffic scenarios. Do not claim a traffic-reduction percentage unless it is calculated from recorded, reproducible runs—and label it as simulated.

9. Scope control: what to build and defer

Feature	Decision for 24-hour MVP	Reason
One school and one gate	Build	Keeps the workflow testable and demoable
Schedule search and constraints	Build	Core planning value
Pickup-zone capacity check	Build a simplified, clearly labelled version	Demonstrates spatial and operational constraints
SUMO baseline and shortlist evaluation	Build early	Provides a credible vehicle-level simulation
Demand surge and low-compliance tests	Build at least one or two	Shows robustness and failure conditions
Parent notice with deterministic verification	Build a template fallback; LLM optional	Completes the workflow without making an LLM a dependency
Multiple schools sharing an arterial road	Defer	Multiplies scheduling, network and coordination complexity
Carpool matching and priority passes	Exclude	Requires family opt-in, coordination and personal data; vehicle reduction cannot be assumed
Automatically generated physical road redesigns	Exclude	Needs verified geometry, engineering judgement and legal approvals
Real-time traffic feeds	Exclude	Adds external dependencies without being necessary for the core workflow
Live beacons, signals or vehicle sensors	Exclude	Contradicts the hardware-free MVP
Parent tracking app	Exclude	The MVP only needs approved notices
Gaussian-process surrogate	Stretch goal	Useful only if simulation evaluation becomes a bottleneck
NSGA-II and full CVaR suite	Stretch goal	Add only after the baseline planner and scenario comparison are reliable

Fallback if SUMO integration fails

Use a simple custom event/queue simulation as a clearly disclosed fallback, but keep testing SUMO from the beginning. Do not present a custom animation as SUMO output. The final demo should accurately label which engine produced each metric.

What makes the MVP complete

- A school configuration can be entered or loaded.
- The baseline can be simulated repeatedly.
- At least one constraint is enforced.
- The planner returns feasible alternatives with objective values.
- A pickup-zone capacity issue is detected.
- Candidate plans are compared using simulation or an explicitly labelled fallback model.
- At least one stress scenario is tested.
- The dashboard explains the recommendation and its limitations.
- The final notice matches the selected plan.

10. Final assessment: uniqueness, judging potential and feasibility

1. Uniqueness

We cannot claim the entire idea is new. School dismissal apps, traffic simulation and staggered scheduling already exist. Our differentiation is combining uncertainty-aware optimisation, pickup-zone capacity, SUMO simulation, family-disruption trade-offs and stress testing in one school-focused workflow.

2. Will judges be impressed?

It has strong potential if you demonstrate real algorithmic results, a working simulation and honest baseline comparisons. No solution guarantees a win.

3. Is it doable in 24 hours?

Yes, as a focused MVP with one school, synthetic traffic data, basic schedule optimisation, SUMO simulation and a working dashboard. The entire advanced architecture is not realistically guaranteed in 24 hours.

4. Can we prove uniqueness?

We can research competitors and demonstrate our specific contribution, but we cannot guarantee that no existing system offers similar capabilities.

My recommendation: Proceed with GateFlow AI, prioritise a fully working end-to-end demo, and prove its value through reproducible results.

Practical positioning

Do not present GateFlow as the first school traffic product or as a guaranteed congestion-reduction system. Present it as a focused integration of constrained planning, uncertain demand, pickup-zone capacity and microscopic simulation—with a measurable, reproducible comparison against the existing schedule and a simple benchmark.

How to discuss uniqueness responsibly

- Research school dismissal products, school transport systems, traffic simulation tools and academic work on staggered scheduling.
- Compare features rather than relying on broad claims that a product does not exist.
- Document the specific workflow and technical combination implemented in the prototype.
- Demonstrate the contribution directly: constraints, objective trade-off, zone capacity, SUMO replay and stress test.
- Use wording such as “our differentiating approach is…” rather than “this has never been done before.”

11. Final pitch to evaluators

GateFlow AI helps school administrators evaluate staggered schedules and pickup-zone arrangements before implementing them. It combines school-specific constraints, uncertain demand estimates, multi-objective planning and microscopic traffic simulation. Rather than promising an unexplained traffic-reduction percentage, GateFlow shows how candidate plans perform under repeatable normal and stressed scenarios, quantifies family disruption, and produces an actionable plan for the principal to review.

The prototype runs without specialised hardware or paid traffic-data APIs. It starts with manual observations or synthetic data, uses open road-network data where available, and is designed to be validated against real observations in a future school pilot.

Final solution definition

Field	Definition
Product	GateFlow AI — School-gate traffic digital twin and decision-support system
User	School principal or transport coordinator
Problem	Arrival/departure peaks can overwhelm available gate-side stopping capacity, creating queues and potential spillback
Inputs	Student groups and bell times, constraints, mapped road context, vehicle proportions, optional manual counts and labelled assumptions
Engine	Demand estimation, fast constrained schedule search, multi-objective comparison, pickup-zone capacity assignment, SUMO simulation and uncertainty/stress testing
Outputs	A small set of feasible plans with timings, zone assignments, simulated metrics, trade-offs and limitations
Action	Principal reviews and approves a plan; GateFlow generates a verified parent notice
Validation	Compare the recommendation with the existing schedule and simple baselines; verify real-world improvement through a future school pilot

Final recommendation

Build GateFlow AI as a single-school, one-gate, uncertainty-aware traffic-planning system. Use the complete product workflow, demand estimation and multi-objective planning, and SUMO-based simulation. Make the basic product work end to end before attempting the Gaussian-process surrogate, full evolutionary optimisation or citywide features.

The most convincing version is the one that can show a judge: here is the existing schedule, here are the constraints, here are the alternatives the algorithm evaluated, here is what the simulator actually produced, and here is why the selected plan is a reasonable choice under these assumptions.

That is a technically substantial and practically demonstrable hackathon solution—without pretending that simulation is proof of real-world traffic reduction.

Appendix: compact demo checklist

- Saved demo network and synthetic dataset are available offline.
- Every data source is labelled observed, synthetic or assumed.
- Baseline and candidate runs use comparable settings and saved seeds.
- Hard constraints and over-capacity zones are tested.

- At least one normal and one stress scenario are shown.
- Charts show exact units and simulated/observed status.
- Parent notice values are checked against approved plan JSON.
- Backup video is ready and the demo has been rehearsed.

Document purpose: solution definition and hackathon implementation plan. Any deployment affecting real school traffic requires local verification, safety/legal review and measured field validation.